技术部第二次技术分享会

Java 基础语法・简明版

姚雨森　技术部分管主席

2026-10-23

输出：让程序主动“开口”

怎样知道程序做了什么？先让它把信息打印到控制台。

```
public class Main {
    public static void main(String[] args) {
        // Print two lines.
        System.out.println("Hello, Java!");
        System.out.println("Let's go shopping!");
    }
}
```

- 示例从 `main` 开始，语句从上到下执行；每次输出后换行。
- `//` 后的注释给人看，不作为语句执行。

后面的代码片段都放在 `main` 的大括号内。

变量：把需要的数据记下来

商品数量可能变化，我们需要保存它，也需要修改它。

```
int quantity = 2;
System.out.println(quantity); // 2

quantity = quantity + 1;
System.out.println(quantity); // 3
```

- `int quantity = 2;`：声明整数变量，并给它一个初始值。
- `=`：先计算右边，再把结果保存到左边的变量中。
- 读取或重新赋值时，直接使用变量名，不再写类型。

数据类型：不同信息有不同表达

商品数量、单价、是否有货和商品名，需要表达不同性质的信息。

要保存什么	常用类型	值的例子
整数数量	`int`	`2`
带小数的数值	`double`	`12.5`
是或否	`boolean`	`true`
单个字符	`char`	`'A'`
一段文本	`String`	`"Notebook"`

先想清楚数据是什么，再选择类型。

数字：整数与小数怎么选

数量通常是整数，测量值、单价等数值可能带小数。

```
int quantity = 2;
double price = 12.5;
long totalVisits = 3000000000L;
```

- 常见整数用 int；需要更大范围时可用 long。
- 小数通常用 double；浮点数可能是近似值。
- 还有 byte、short、float，按具体需求再了解。

类型能表示的范围和精度有限，不需要现在背下全部数值。

boolean：表达“是或否”

商品有没有货？用户是不是会员？这些状态只有两种答案。

```
boolean inStock = true;
boolean isMember = false;

System.out.println(inStock);  // true
System.out.println(isMember); // false
```

- `true` 表示真，`false` 表示假。
- `boolean` 只有这两个值，不用数字 0 和 1 代替。

后面可以根据这些状态，让程序采取不同的动作。

字符与字符串：保存文字

一个等级字母可以用 char，商品名称可以用 String。

```
char grade = 'A';
String name = "Notebook";

System.out.println(name);   // Notebook
System.out.println("name"); // name
```

- 字符用单引号，如 'A'；文本用双引号，如 "Notebook"。
- name 读取变量的值，"name" 表示文字本身。
- String 首字母大写，不属于基本数据类型。

运算：从单价和数量得到总价

买两种商品：分别计算金额，再把结果加起来。

```
double priceA = 12.5;
double priceB = 8.0;
int quantityA = 2;
int quantityB = 3;
double total = priceA * quantityA + priceB * quantityB;
System.out.println(total); // 49.0
```

- `+ - * /` 表示加减乘除，`%` 求余数；括号中的部分先算。
- 整数相除：`7 / 2` 得到 3；`7.0 / 2` 得到 3.5。
- 也可先转换类型：`(double) 7 / 2` 得到 3.5。

字符串拼接：让结果看得懂

只看到一个数字还不够，我们可以给结果加上说明。

```
String name = "Notebook";
double total = 25.0;
System.out.println(name + " total: " + total);
```

输出： Notebook total: 25.0

+ 遇到字符串时进行拼接。想先计算，就加括号：

```
System.out.println("Count: " + (2 + 3)); // Count: 5
```

条件：把规则变成真假判断

“总价超过 40 元，并且是会员”，可以写成条件表达式。

```
double total = 49.0;
boolean isMember = true;
boolean eligible = total > 40 && isMember;
System.out.println(eligible); // true
```

- 比较：`>` `>=` `<` `<=`，以及相等 `==`、不等 `!=`。
- 组合：`&&` 表示“并且”，`||` 表示“或者”，`!` 表示“取反”。
- 判断的结果是 `true` 或 `false`。

注意：`=` 是赋值；`==` 才是相等判断。

if / else：根据情况选择动作

新需求：总价超过 40 元就打八折，否则保持原价。

```
double total = 49.0;
if (total > 40) {
    total = total * 0.8;
    System.out.println("Discount applied");
} else {
    System.out.println("No discount");
}
System.out.println("Total: " + total);
```

- 条件成立执行 `if` 分支，否则执行 `else` 分支。
- 大括号圈出同一分支的操作；只需“满足才做”时，可以省略 `else`。

else if：多个情况选择一种

新需求：根据总价，把订单分成三个档次。

```
double total = 80.0;
if (total > 100) {
    System.out.println("Large order");
} else if (total > 40) {
    System.out.println("Medium order");
} else {
    System.out.println("Small order");
}
```

- 从上到下判断，只执行**第一个成立条件**对应的分支。
- 本例输出 `Medium order`；所有条件都不成立时执行 `else`。

while：一次不够，就继续做

模拟余额不足时每次增加 50 元，直到足够支付 180 元。

```
int balance = 50;
int total = 180;
while (balance < total) {
    balance = balance + 50;
}
System.out.println(balance); // 200
```

余额：　$50 \to 100 \to 150 \to 200$　停止

- 每轮先判断；条件成立就执行，再回到条件处。
- 一开始就足够时，一次也不执行；余额始终不变则可能一直循环。

for：有规律地重复

新需求：依次打印第 1～5 件商品的编号。

```
for (int number = 1; number <= 5; number++) {
    System.out.println("Item: " + number);
}
```

1. 起点：`int number = 1`，只在开始时执行一次。
2. 条件：`number <= 5`，每轮开始前检查。
3. 更新：`number++`，每轮执行完后将编号加一。

先初始化，再反复执行“判断 → 循环体 → 更新”。

分支与循环：什么时候用哪一个

需求	常用写法	购物中的例子
根据条件选择动作	`if / else`	满足条件才打折
多个情况选择一种	`else if`	按金额划分档次
条件成立就继续	`while`	余额不足就继续增加
按计数规律重复	`for`	打印第 1～5 件商品编号

`while` 和 `for` 常能互相改写，选择能把过程表达清楚的写法。

分支决定“做哪一件”，循环决定“是否继续做”。

循环 + 判断：筛选符合要求的数字

小练习：打印 1～1000 中能被 7 整除的数。

```
for (int number = 1; number <= 1000; number++) {
    if (number % 7 == 0) {
        System.out.println(number);
    }
}
```

- `for`：依次检查 1～1000。
- `number % 7 == 0`：除以 7 的余数为 0。
- `if`：只打印符合条件的数。

输出：7、14、21、……、994，每个数占一行。

再加一个变量，就能统计数量

新需求：除了打印，还要知道一共有多少个。

```
int count = 0;
for (int number = 1; number <= 1000; number++) {
    if (number % 7 == 0) {
        System.out.println(number);
        count++;
    }
}
System.out.println("Count: " + count); // Count: 142
```

- count 在循环前初始化，每找到一个就加一。
- 如果每轮都把它重设为 0，就无法保留前面累计的结果。

遇到问题时，想到这些工具

我想做什么	可以用什么
看见程序的结果	输出
保存和更新数据	变量
表达不同性质的信息	数据类型
计算数值、组织文字	运算、字符串拼接
根据情况执行不同操作	分支
重复执行一段操作	循环

先想清楚数据、条件和重复过程，再查具体写法。